BIL 322 · DÖNEM PROJESİ · FAZ 1 LABORATUVARI

Otonom Drone Sürüsü Koreografları

Thread’ler, kuyruklar, kilitler, barrier’lar ve bataryası biten bir kaptan.



PROJE

Müzikle dans eden bir drone sürüsü

Yüzlerce LED drone bir koreografi sergiler, sonra gökyüzünde bir resim oluşturur. Gösteriyi dört rol yapar.

Merkez

Sanat Yönetmeni

Notayı ve hedef resmi okur, her bölümün temasını belirler, beat saatini çalıştırır, yavaş planlayıcıyı barındırır.

Leader'lar

Dans Kaptanları

Her bölge için bir süreç. Her biri plan ister, planı doğrular ve topluluğunu yönetir.

Follower'lar

Dansçılar

Faz 1'de bir leader'ın içindeki thread'ler. Her biri bir hücreye uçar, bir renk yakar, rapor verir.

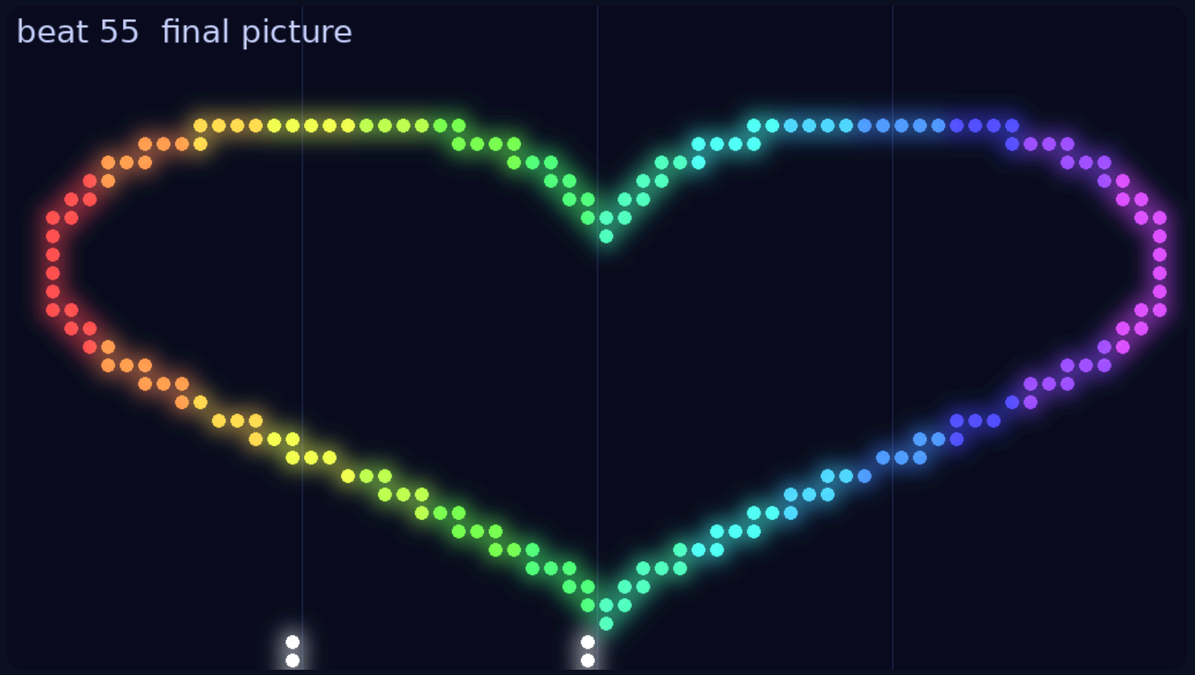
Viewer

Yayın Kabini

Faz 2'den itibaren ayrı bir GUI. Kare atlayabilir, sürüyü asla yavaşlatmaz.

HEDEF

Çalışınca nasıl görünür



4 kaptan × 46 dansçı = **184 drone**. Her nokta bir drone’dur, kalbin her pikseline bir drone düşer.

Alttaki 4 beyaz nokta, evde kalan yedek drone’lardır.

```
./center --score data/scores/show.score --
followers 46 --ascii 8
```

Faz 2 bunu gerçek bir pencerede çizer.

ÜÇ FAZ, TEK SİSTEM

Aynı protokol, üç taşıma yolu

	Faz 1 (bu hafta)	Faz 2	Faz 3
Leader	child süreç (fork, exec)	child süreç	client süreç
Dansçı	kaptanın içinde thread	kaptanın içinde thread	ayrı süreç
Bağlantılar	pipe'lar	pipe'lar + paylaşımlı bellek	TCP socket'ler
Viewer	terminalde metin	ayrı süreç, paylaşımlı bellek	socket client
Kaptan ölünce	center fark eder ve yeniden başlatır	+ hücreleri serbest bırakılır	yetim kalanlar yeniden yerleştirilir

Gönderme ve alma kodunu **file descriptor**'lar üzerine yaz ve ayrıştırmayı (parsing) taşıma yolundan ayrı tut: Faz 3'te programı değil, yalnızca taşıma yolunu değiştirirsin.

FAZ 1 MİMARİSİ

İki tür süreç, çok sayıda thread

center

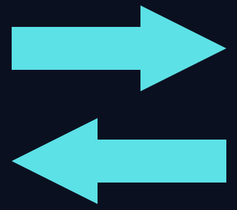
tek süreç, parent

main thread · saat döngüsü: THEME, BEAT, READY ve ACK bekler, sahneyi denetler

reader thread'ler · her leader için bir tane, pipe'ını ayrıştırır

planner havuzu · sınırlı bir istek kuyruğunun arkasında thread'ler, Lab 6 havuzu + arena

pipe'lar



mesaj başına bir metin satırı

leader × 4

bölge başına bir child süreç: fork, dup2, exec

main thread · center'ın satırlarını okur

thinker · PLAN_REQ, doğrulama, READY

follower thread'ler · F0 … Fk, beat başına bir barrier, bölgenin doluluk ızgarasında hücre talep eder

PROTOKOL (SABİT)

Satır başına bir mesaj, düz metin

center → leader

`ASSIGN id x0 y0 x1 y1 n`	senin bölgen
`THEME sec start end theme mood`	genel yön
`PLAN req n`	ardından n keyframe satırı
`BUSY req retry_ms`	kuyruk dolu, sonra dene
`BEAT n`	n. beat'i başlat
`SHUTDOWN`	hoşça kal

leader → center

`HELLO id n`	buradayım
`PLAN_REQ req sec n seed`	bana bir plan ver
`READY id sec`	plan yüklendi
`POS id beat f x y c`	f nerede
`ACK id beat`	beat bitti

- Keyframe satırı: **`beat follower x y color`**. Bir plan, her beat ve her follower için bir tane içerir.
- Bilinmeyen mesaj: logla ve yok say. Bozuk mesaj: reddet, asla çökme. Satırlar en fazla 256 byte.
- Bu satırları stdin'ine elle yazarak bir leader'ı test edebilirsin.

NASIL ÇALIŞIR

Bir beat, adım adım

1. Center **BEAT b** gönderir (ancak b−1 için ACK geldikten sonra).
2. Her follower, kayıtlı plandan b için hedefini bulur (plan yoksa: olduğu yerde kalır).
3. Bulunduğun hücreyi **bırak (release)**, sonra **barrier**'da bekle.
4. Hedef hücreyi ızgarada **talep et (claim)**, sonra tekrar **barrier**.
5. Barrier'ın seçtiği thread **POS** satırlarını ve **ACK b**'yi gönderir.
6. Center çarpışmaları ve hızı denetler, sonra BEAT b+1 gönderir.

Neden iki barrier?

Bir beat içinde A follower'ı, B follower'ının terk ettiği hücreye girebilir.

Tüm bırakmalar herhangi bir talepten önce olmalı, yoksa A'nın talebi yanlışlıkla başarısız olur.

Başarısız bir talep bir güvenlik ihlalidir; döngüde tekrar denenecek bir şey değildir.

YAPMAN GEREKENLER

İskelette altı görev

#	Görev	Neresi	Puan
1	Sınırlı kuyruk	`queue.c`	20%
2	Koreograf ve kaptan	`center.c, leader.c (thinker)`	25%
3	Dansçılar ve ızgara	`grid.c, leader.c (follower)`	25%
4	Deadlock'u bul	`grid_swap, swapstress`	10%
5	Kaptan düştü	`center.c (on_leader_exit)`	10%
6	Testler ve rapor	`make test, make tsan, REPORT.md`	10%

Hazır verilenler: protokol ayrıştırıcısı, planlayıcı, doğrulayıcı, yükleyiciler, saat döngüsü, birim testleri ve test betiği. **Senin işin:** TODO blokları.

GÖREV 1 · %20

Sınırlı kuyruk

```
queue_t *queue_create(size_t cap, size_t item_size);
int queue_push(queue_t *q, const void *item); // doluysa bekler
int queue_try_push(queue_t *q, const void *item); // doluysa -1
int queue_pop(queue_t *q, void *out); // boşsa bekler
void queue_close(queue_t *q); // herkesi uyandırır
size_t queue_size(queue_t *q); void queue_destroy(queue_t *q);
```

İki yerde kullanılır: center'ın **istek kuyruğu** (try_push, doluysa BUSY) ve leader'ın tema kuyruğu.

Kurallar

- Her pthread_cond_wait'in etrafında if değil **while**
- close'dan sonra: pop kalanları boşaltır, sonra −1 döner; push −1 döner
- close **herkesi** uyandırır: broadcast
- Meşgul bekleme (busy-waiting) yok, sleep yok

GÖREV 2 · %25

Koreograf ve kaptan

Center

handle_plan_req, serve_request

- try_push; kuyruk doluysa: **BUSY 50**. Reader thread asla beklemez
- **Semafor**: en fazla --max-inflight planlayıcı çağrısı
- KV blokları Lab 6 havuzundan. Havuz **thread-safe değildir**: pool_mu ile sar
- Havuz boşsa: BUSY. Bu da bir geri basınçtır (backpressure)
- Her çıkış yolu blokları geri verir ve semaforu bırakır (post)

Kaptan

thinker_main

- Her THEME için: **PLAN_REQ** gönder, PLAN veya BUSY bekle
- BUSY: retry_ms + **rastgele** bir ek süre uyu. Sabit gecikme tüm leader'ları aynı anda tekrar denetir: livelock
- **validate_plan** çalıştır. Geçersizse: yeni seed, en fazla 6 deneme (--flaky 50 bunu test eder)
- Vazgeçsen bile her zaman **READY** gönder: topluluk yerinde bekler ve gösteri sürer

GÖREV 3 · %25

Birlikte adım atan dansçılar ve ızgara

Doluluk ızgarası

grid_claim, grid_release, grid_owner

- Hücre başına bir drone, her leader'ın bölgesinde
- İki mod: ızgara için **tek mutex** veya **satır başına bir mutex**
- Okuyucu da kilitlemeli. Neden?
- make bench ikisini 1, 2, 4, 8 thread ile ölçer. Tabloyu rapora koy ve **açıkla**

follower_main

sırayla her beat b için

```
BEAT b'yi bekle
hedef = plan veya bekle
release(eski hücre)
barrier
claim(hedef)
barrier
serial thread: POS satırları, ACK
```

Başarısız claim = güvenlik ihlali: say, logla, olduğun yerde kal.

GÖREV 4 VE 5 · HER BİRİ %10

Bulunacak bir deadlock, kaybedilecek bir kaptan

4 · Deadlock’u bul

```
./swapstress row 8 2
```

- grid_swap iki satırı kilitler. Satır modunda **takılır**
- Takılan thread’leri göster: gdb -p, thread apply all bt
- Hangi Coffman koşulunu bozduğunu söyle; kilit sıralamasıyla düzelt
- Çözümünün neden livelock’a yol açmadığını açıkla

5 · Kaptan düştü

```
./center --battery 1:20
```

- Poll ile değil, **pipe üzerindeki EOF** ile fark et. “captain 1 went dark” yazdır
- **waitpid**, pipe’ları kapat: zombie yok
- Saat döngüsünü (st_cv) uyandır ki beklemeyi bıraksın
- Yeniden başlat: ASSIGN, sonraki beat’ten itibaren THEME; topluluk evde başlar

GÖREV 5, OLMASI GEREKEN GÖRÜNÜM

Kaptan 1 ölür, gösteri sürer



Beat 24: ikinci topluluk evine döndü (beyaz): yeni kaptan, yeni thread'ler.



Beat 25: yeni kaptanın planı başlar ve topluluk diğerleriyle dans eder.



Beat 55: resim yine oluşur; sıfır çarpışma ve sıfır hız ihlaliyle.

BAŞLARKEN

Derle, test et ve çalışma sırası

```
cd labs/swarm-phase1-skeleton
make # derler, TODO'lar boş
make test # her kontrol için PASS/FAIL
make tsan # WARNING olmamalı
make bench # tek kilit ve satır kilitleri
./center --tempo 0 --ascii 8
```

Tam gösteri Görev 1–3'ün birlikte çalışmasını ister. O zamana kadar her parçayı tek başına test et: **test_queue**, **swapstress**, **gridbench**, veya protokol satırlarını elle yaz: **./leader --id 0**.

Önerilen sıra

1. queue.c
2. grid claim, release, owner
3. center: serve_request
4. leader: thinker
5. leader: followers
6. kaptanı yeniden başlatma
7. deadlock düzeltmesi

TESLİM VE NOTLANDIRMA

Ne teslim edilir, ne puan kırar

Teslim

- Repository'n (GitHub Classroom) ve Google Classroom'daki bağlantı
- **make test** ve **make tsan** geçer; Valgrind temiz
- tests/run_tests.sh içinde en az **iki kendi testin**
- **REPORT.md**: mimari taslağı, her mutex'in neyi koruduğu, kilit sıralaması, deadlock açıklaması, kaptan düşmesinin ele alınışı, bench tablosu, yapay zekâ yardımı

Sıfır tolerans

- Meşgul bekleme (busy-waiting)
- pthread_cond_wait etrafında **if**
- Korunmayan Lab 6 havuzu
- Kaptan ölünce kalan zombie süreçler
- Kapanışta takılma
- Eksik rapor

Bonus en fazla +%10: öncelikli kuyruk, ızgara için reader-writer lock, şarj istasyonu semaforlu batarya sayacı.

TAKILDIĞINDA

Dört tuzak ve nasıl görülür

Kimsenin ulaşmadığı barrier

Barrier **tüm** follower'lar tarafından paylaşılır. İki bekleme arasında tek bir erken return veya continue, herkesi sonsuza kadar bekletir.

Hiç bırakılmayan kilit

Her unlock her yolda olmalı. Erken return'ler olağan sızıntıdır: kuyrukta ve serve_request'te.

Yanlış kilit sırası

Center'da: önce wlock, sonra st_mu. Aynı kural grid_swap'ı düzeltir: her zaman önce küçük numaralı satır.

Yarışı sleep ile gizlemek

Asla. Bunun yerine make tsan çalıştır ve yazdırdığı iki stack'i oku.

Takılma, --timeout saniye sonra **center: TIMEOUT** yazdırır. Şununla bağlan: `gdb -p PID`, sonra `thread apply all bt`.